

TRAINEE LAB GUIDE

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc.

Lab 3.6

Wrap Shopify APIs as Claude Tools (Node.js)

MODULE 3 CAPSTONE LAB

Module	3 — Shopify Platform, Storefront API, Admin API & Native AI
Topic	3.6 — a complete Claude-powered Shopify agent, built end to end
Estimated time	60–75 minutes
Tools needed	Terminal, Node.js 20+, a code editor, an active Claude API key
Prerequisite	A Shopify dev store, with a Headless-channel Storefront private token and a Dev Dashboard custom app's Client ID/secret (scoped for <code>write_draft_orders</code>) already in hand

What You're Building

GOAL

A complete, standalone Claude agent — built from nothing in this one lab — with three tools that call your real Shopify dev store: searching products, building a cart, and creating a draft order. No mock data anywhere, and no MCP yet (that's Module 7's job).

The three tools deliberately exercise two different kinds of Shopify credential in the same agent: two tools run on a read-only Storefront token, one runs on a write-capable Admin token — a real access-control boundary, not just a naming convention.

The three tools

- `search_products` — Storefront API, read-only
- `create_cart_with_item` — Storefront API, read-only token, but a write action on the cart
- `create_draft_order` — Admin API, write-capable, gated behind an explicit y/n confirmation before it creates anything

Prefer to vibe-code this instead? The Module 3 page has a ready-to-use Claude Code prompt covering this same build. This guide gives you the concrete code either way — to type yourself, or to compare against what Claude Code produces.

Steps 1–3: Project, Credentials, API Clients

1 Create the project and install dependencies

TERMINAL

```
mkdir shopify-claude-agent
cd shopify-claude-agent
npm init -y
npm install @anthropic-ai/claude-agent-sdk zod dotenv
```

Open the generated `package.json` and add `"type": "module"` at the top level, so `import / export` syntax works without a build step.

.GITIGNORE

```
node_modules
.env
```

2 Add credentials to `.env`

```
ANTHROPIC_API_KEY=your-claude-api-key
SHOPIFY_STORE_DOMAIN=your-store.myshopify.com
STOREFRONT_PRIVATE_TOKEN=shpat_your-private-token
SHOPIFY_CLIENT_ID=your-client-id
SHOPIFY_CLIENT_SECRET=your-client-secret
```

Variable	Where it comes from
ANTHROPIC_API_KEY	Your Claude API key, from the Anthropic Console
SHOPIFY_STORE_DOMAIN	Your dev store's <code>.myshopify.com</code> domain
STOREFRONT_PRIVATE_TOKEN	The store's Headless sales channel → Create storefront → private access token — read-only, long-lived
SHOPIFY_CLIENT_ID / SHOPIFY_CLIENT_SECRET	A custom app registered in the store's Dev Dashboard , scoped for <code>write_draft_orders</code> — exchanged programmatically for a short-lived Admin token

Two credential types, on purpose: the Storefront token can only ever read/build a cart; the Admin credentials are the only path to writing a draft order. Keeping them separate is a real access-control boundary this lab exercises deliberately, not incidental setup.

3 Write the two Shopify API clients

These wrap every GraphQL call the agent will make — one file per credential type:

STOREFRONTCLIENT.JS

```
const domain = process.env.SHOPIFY_STORE_DOMAIN;
const token = process.env.STOREFRONT_PRIVATE_TOKEN;

export async function storefrontRequest(query, variables) {
  const res = await fetch(`https://${domain}/api/2026-07/graphql.json`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Shopify-Storefront-Private-Token": token,
    },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors) throw new Error(JSON.stringify(json.errors));
}
```

```
    return json.data;
  }
}
```

ADMINCLIENT.JS

```
const domain = process.env.SHOPIFY_STORE_DOMAIN;
const clientId = process.env.SHOPIFY_CLIENT_ID;
const clientSecret = process.env.SHOPIFY_CLIENT_SECRET;

let cachedToken = null;
let cachedTokenExpiresAt = 0;

async function getAdminAccessToken() {
  if (cachedToken && Date.now() < cachedTokenExpiresAt) return cachedToken;
  const res = await fetch(`https://${domain}/admin/oauth/access_token`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      client_id: clientId, client_secret: clientSecret, grant_type: "client_credentials",
    }),
  });
  const data = await res.json();
  if (!data.access_token) throw new Error("Admin token exchange failed: " + JSON.stringify(data));
  cachedToken = data.access_token;
  cachedTokenExpiresAt = Date.now() + (data.expires_in - 60) * 1000;
  return cachedToken;
}

export async function adminRequest(query, variables) {
  const token = await getAdminAccessToken();
  const res = await fetch(`https://${domain}/admin/api/2026-07/graphql.json`, {
    method: "POST",
    headers: { "Content-Type": "application/json", "X-Shopify-Access-Token": token },
    body: JSON.stringify({ query, variables }),
  });
  const json = await res.json();
  if (json.errors) throw new Error(JSON.stringify(json.errors));
  return json.data;
}
```

Why the Admin client caches its token: the client-credentials exchange returns a token that expires in 24 hours. Requesting a fresh one on every single call would work, but caching until just before expiry avoids the extra round trip on every tool call.

Steps 4–6: Build the Three Tools

4 Write `search_products`

Read-only, Storefront-backed. Returns a real variant ID with every result, so later tools never have to guess where one comes from:

TOOLS/SEARCHPRODUCTS.JS

```
import { tool } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";
import { storefrontRequest } from "../storefrontClient.js";

export const searchProducts = tool(
  "search_products",
  "Search the store catalog for products matching a text query.",
  { query: z.string().default("") },
  async ({ query }) => {
    const data = await storefrontRequest(`
      query searchProducts($query: String) {
        products(first: 5, query: $query) {
          edges {
            node {
              title
              priceRange { minVariantPrice { amount currencyCode } }
              variants(first: 1) { edges { node { id } } }
            }
          }
        }
      }`, { query });

    const output = z.object({
      products: z.array(z.object({
        title: z.string(), price: z.string(), currency: z.string(), variantId: z.string(),
      })),
    }).parse({
      products: data.products.edges.map((e) => ({
        title: e.node.title,
        price: e.node.priceRange.minVariantPrice.amount,
        currency: e.node.priceRange.minVariantPrice.currencyCode,
        variantId: e.node.variants.edges[0]?.node.id ?? "",
      })),
    });

    return { content: [{ type: "text", text: JSON.stringify(output) }] };
  }
);
```

A real, testable pitfall: pass `query: ""` when there's no real search term (a shopper asking "what do you sell?"), never `null` — on Shopify's Storefront API, an empty string returns the unfiltered catalog, while `null` returns zero

products. The `z.string().default("")` schema above bakes that in.

5 Write `create_cart_with_item`

Also Storefront-backed — creates a cart, adds one line item, and returns a real checkout link:

TOOLS/CREATECARTWITHITEM.JS

```
import { tool } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";
import { storefrontRequest } from "../storefrontClient.js";

export const createCartWithItem = tool(
  "create_cart_with_item",
  "Create a cart and add one product variant to it. Returns a checkout link.",
  { variantId: z.string(), quantity: z.number().int().positive().default(1) },
  async ({ variantId, quantity }) => {
    const created = await storefrontRequest(
      `mutation { cartCreate { cart { id } } }`
    );

    const added = await storefrontRequest(
      `mutation cartLinesAdd($cartId: ID!, $lines: [CartLineInput!]) {
        cartLinesAdd(cartId: $cartId, lines: $lines) {
          cart { checkoutUrl }
          userErrors { field message }
        }
      }`,
      { cartId: created.cartCreate.cart.id, lines: [{ merchandiseId: variantId, quantity } ] }
    );

    if (added.cartLinesAdd.userErrors.length) {
      return {
        content: [{ type: "text", text: JSON.stringify(added.cartLinesAdd.userErrors) }],
        is_error: true,
      };
    }

    const output = z.object({ checkoutUrl: z.string().url() })
      .parse({ checkoutUrl: added.cartLinesAdd.cart.checkoutUrl });
    return { content: [{ type: "text", text: JSON.stringify(output) } ] };
  }
);
```

6 Write `create_draft_order` and its confirmation gate

The only write-capable, Admin-backed tool — and the only one that can't run without an explicit human "yes." First, a small shared confirmation helper:

CONFIRM.JS

```
import readline from "node:readline/promises";

export const rl = readline.createInterface({ input: process.stdin, output: process.stdout });

export async function askConfirmation(question) {
  const answer = await rl.question(question);
  return answer.trim().toLowerCase() === "y";
}
```

TOOLS/CREATEDRAFTORDER.JS

```
import { tool } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";
import { adminRequest } from "../adminClient.js";
import { askConfirmation } from "../confirm.js";

export const createDraftOrder = tool(
  "create_draft_order",
  "Create a draft order for a product variant and return its invoice link. " +
  "Requires the user's explicit y/n confirmation before it creates anything.",
  { variantId: z.string(), quantity: z.number().int().positive().default(1) },
  async ({ variantId, quantity }) => {
    const confirmed = await askConfirmation(
      `Create a draft order for ${quantity}x ${variantId}? (y/n) `
    );
    if (!confirmed) {
      return {
        content: [{ type: "text", text: "Declined: the draft order was not created." }],
        is_error: true,
      };
    }

    const data = await adminRequest(`
      mutation createDraftOrder($variantId: ID!, $quantity: Int!) {
        draftOrderCreate(input: { lineItems: [{ variantId: $variantId, quantity: $quantity }] }) {
          draftOrder { id invoiceUrl }
          userErrors { field message }
        }
      }
    `, { variantId, quantity });

    if (data.draftOrderCreate.userErrors.length) {
      return {
        content: [{ type: "text", text: JSON.stringify(data.draftOrderCreate.userErrors) }],
        is_error: true,
      };
    }

    const output = z.object({ invoiceUrl: z.string().url() })
      .parse({ invoiceUrl: data.draftOrderCreate.draftOrder.invoiceUrl });
    return { content: [{ type: "text", text: JSON.stringify(output) }] };
  }
);
```

```
}  

```

This confirmation is the whole point of the tool. Without it, any message that resembles a purchase request could trigger a real write against your store's orders. Never let a write-capable tool skip this gate, here or elsewhere.

Steps 7–8: Assemble the Agent and Run It

7 Write `agent.js`

This registers the three tools as one in-process MCP server, then runs an interactive loop that keeps the conversation alive across turns:

AGENT.JS

```
import "dotenv/config";
import { query, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
import { searchProducts } from "../tools/searchProducts.js";
import { createCartWithItem } from "../tools/createCartWithItem.js";
import { createDraftOrder } from "../tools/createDraftOrder.js";
import { rl } from "../confirm.js";

const shopifyServer = createSdkMcpServer({
  name: "shopify-tools",
  tools: [searchProducts, createCartWithItem, createDraftOrder],
});

let turnComplete = () => {};
let turnCompletePromise = new Promise((resolve) => { turnComplete = resolve; });

async function* userMessages() {
  console.log("Shopify agent ready. Type a message (Ctrl+C to quit).\n");
  while (true) {
    const text = await rl.question("You: ");
    yield { type: "user", message: { role: "user", content: text } };

    // Wait for the SDK to signal this turn is fully done - including any
    // tool-side confirmation prompt - before looping back to ask again.
    // Otherwise this rl.question("You: ") races askConfirmation's own
    // rl.question() call inside createDraftOrder, and the real answer
    // (and the resulting invoiceUrl) gets silently swallowed.
    await turnCompletePromise;
    turnCompletePromise = new Promise((resolve) => { turnComplete = resolve; });
  }
}

async function main() {
  const stream = query({
    prompt: userMessages(),
    options: {
      mcpServers: { shopify: shopifyServer },
      allowedTools: [
        "mcp__shopify__search_products",
        "mcp__shopify__create_cart_with_item",
        "mcp__shopify__create_draft_order",
      ],
      systemPrompt:
        "You are a helpful Shopify shopping and order assistant. " +
        "Use the available tools for anything involving real product, cart, or order data.",
    },
  });
}
```

```

    },
  });

  for await (const message of stream) {
    if (message.type === "assistant") {
      const text = message.message.content
        .filter((block) => block.type === "text")
        .map((block) => block.text)
        .join("");
      if (text) console.log("Agent:", text, "\n");
    }
    if (message.type === "result") {
      turnComplete();
    }
  }
}

main();

```

Without `allowedTools`, the agent silently hangs. `query()` 's default permission mode asks for approval before every tool call — but this script never wires up a `canUseTool` handler to answer that prompt, so it just stalls forever the first time a tool would run, with no visible error. Listing the three tools under `allowedTools` (using the `mcp__<serverName>__<toolName>` naming the SDK expects — here, server name `shopify`) pre-approves them so the loop actually proceeds. This doesn't loosen safety anywhere: `create_draft_order` still runs its own y/n confirmation inside the tool itself regardless of this setting, so a real write still can't happen without an explicit human "yes."

A second, subtler race — the confirmation prompt and the next "You:" prompt share one `readline` interface. Without the `turnComplete` gate above, `userMessages()` loops straight back to `r1.question("You: ")` the instant it yields — it has no way to know the current turn is still in flight. If that turn happens to call `create_draft_order`, its own `askConfirmation` prompt inside `tools/createDraftOrder.js` also calls `r1.question()` on that same shared interface, and the two calls race. The symptom isn't an error — it's the agent silently skipping past your `y` and jumping straight to the next `You:` prompt, with no `invoiceUrl` ever shown. Waiting on the stream's `"result"` message — the SDK's own signal that a turn, including any tool-side prompts, has fully finished — before yielding again closes that gap.

Where the memory comes from: passing an async generator as `prompt` keeps a single `query()` session open across every line typed into `userMessages()` — the SDK carries conversation state for you inside that one session, which is why a later turn can reference an earlier one without you managing a messages array by hand.

8 Run the full conversation

TERMINAL

```
node agent.js
```

Work through this transcript, one line at a time:

You: Do you have any snowboards?

→ *calls `search_products`; real titles and prices come back from your dev store, with no permission prompt in the way*

You: Add one to my cart

→ *calls `create_cart_with_item` using the `variantId` from the search result; returns a real `checkoutUrl`*

You: Actually, create a draft order for it instead

→ *calls `create_draft_order`; the agent prints the y/n confirmation prompt and waits — type y and confirm the actual `invoiceUrl` comes back in the agent's own reply (not a silent jump to the next You: prompt), or n to see it declined cleanly*

What you'll see: real Shopify data at every step — actual product titles, a working checkout link, and (after confirming) the agent's final response containing a working invoice link. Nothing here is mocked, and only the draft-order step ever pauses for your input.

Acceptance checklist

- ☐ All three tools call the real Shopify GraphQL APIs, not mock data
- ☐ The Storefront token and the Admin client credentials are kept in separate files, matching each tool's actual needs
- ☐ The Admin token-exchange helper refreshes automatically rather than reusing a stale token past its 24-hour expiry
- ☐ `search_products` and `create_cart_with_item` run immediately, with no permission prompt blocking them
- ☐ The draft-order tool still pauses for a real confirmation before creating anything
- ☐ Confirming with `y` makes the real `invoiceUrl` appear in the agent's own reply — not a silent jump straight to the next `You:` prompt
- ☐ A natural-language conversation can search a product, add it to a cart, and produce a real invoice link, all against your dev store

Troubleshooting

Symptom	Likely cause	Try this
The agent just hangs after a message that should trigger a tool — no error, no reply	<code>allowedTools</code> is missing from <code>query()</code> 's options, so the SDK's default permission prompt has no <code>canUseTool</code> handler to answer it and waits forever	Add the three qualified tool names to <code>allowedTools</code> as shown in Step 7 — <code>mcp__shopify__search_products</code> etc.
Typing <code>y</code> at the draft-order confirmation silently jumps to the next <code>You:</code> prompt — no <code>invoiceUrl</code> ever appears	<code>userMessages()</code> looped back to <code>r1.question("You:")</code> before the current turn finished, racing <code>askConfirmation</code> 's own <code>r1.question()</code> call inside <code>tools/createDraftOrder.js</code> on the same shared <code>readline</code> interface	Add the <code>turnComplete</code> gate from Step 7 — the generator must <code>await</code> the stream's "result" message before yielding the next user message, not loop back immediately
A generic question like "what do you sell?" returns no products	<code>query</code> defaulted to <code>undefined</code> instead of an empty string	Confirm the zod schema is <code>z.string().default("")</code> , and that the GraphQL variable actually receives <code>""</code> , not <code>null</code>
<code>create_cart_with_item</code> fails with a merchandise/variant error	A product ID was used instead of a variant ID	<code>variantId</code> must be the <code>ProductVariant</code> <code>gid</code> <code>search_products</code> already returns, not the product's own <code>id</code>
<code>create_draft_order</code> never asks for confirmation	The <code>askConfirmation</code> call is missing, or was placed after the <code>adminRequest</code> call instead of before it	Re-check the tool body against Step 6 — confirmation must run and be checked before any request is sent
Everything fails with an Admin authentication error after the agent's been running a while	The cached Admin token expired and the refresh logic isn't being hit	Confirm <code>adminRequest</code> always calls <code>getAdminAccessToken()</code> first — never a cached token variable directly
A tool throws a zod validation error on its own output	The GraphQL response shape doesn't match what the tool assumed (e.g. a renamed field)	Log the raw response before parsing it with zod, and compare field-by-field against the query
The agent never seems to remember earlier turns	A new <code>query()</code> call is being started per message instead of one long-lived session fed by the async generator	Confirm <code>main()</code> calls <code>query()</code> exactly once, with <code>userMessages()</code> as its <code>prompt</code> — not once per line typed

Where This Goes Next

Keep this project. Module 4's end-to-end shopping agent builds directly on `search_products` and `create_cart_with_item` — the customer-facing purchase flow never uses draft orders. `create_draft_order`'s real home is Module 5's separate, locked-down admin agent, where write-capable tools like this one get their own dedicated, password-protected surface rather than sitting in the same agent a shopper talks to. Module 7 later exposes this same tool set over the network as a standards-compliant MCP server.

Shopify Agentic AI Developer Bootcamp — Koenig Solutions for V Group Inc. · Module 3: Shopify Platform, Storefront API, Admin API & Native AI · Full module content: see the program website, Module 3 page.